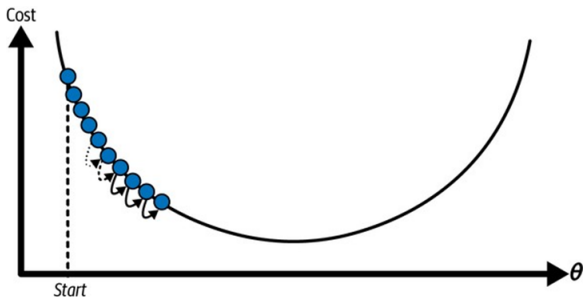
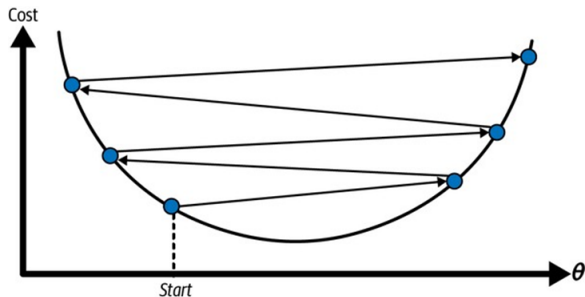


Minh họa: Tốc độ học quá nhỏ và quá lớn



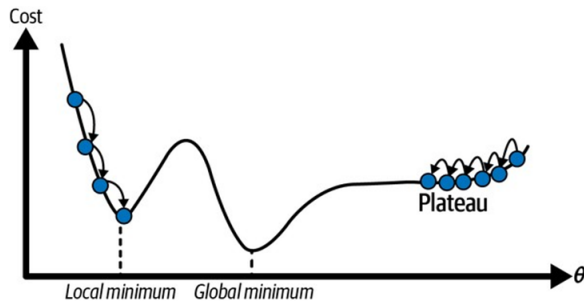
Hình 4-4. Tốc độ học quá nhỏ



Hình 4-5. Tốc độ học quá lớn

Các cạm bẫy của Gradient Descent

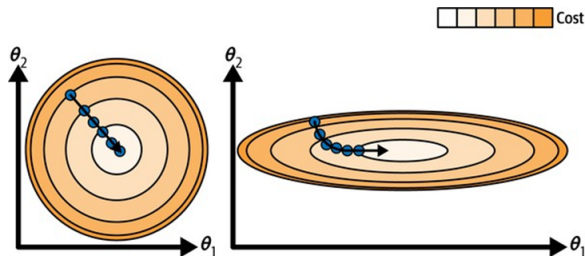
- Không phải tất cả các hàm chi phí đều trông giống như những cái bát hình parabol hoàn hảo.
- **Cực tiểu cục bộ (Local Minimum):** Nếu bạn bắt đầu từ một điểm khởi tạo ngẫu nhiên, thuật toán có thể hội tụ về điểm thấp cục bộ chứ không phải cực tiểu toàn cục (Global Minimum).
- **Cao nguyên (Plateau):** Hàm chi phí đi ngang, khiến thuật toán mất rất nhiều thời gian để đi qua và có thể bị dừng lại sớm.
- Rất may, hàm chi phí MSE của hồi quy tuyến tính là **hàm lồi (convex function)**, đảm bảo không có cực tiểu cục bộ.



Hình 4-6. Cạm bẫy cực tiểu cục bộ

Tầm quan trọng của việc điều chỉnh đặc trưng (Scaling)

- Khi sử dụng Gradient Descent, bạn phải đảm bảo rằng tất cả các đặc trưng có thang đo (scale) tương tự nhau (VD: dùng StandardScaler).
- **Bên trái:** Các đặc trưng được chia tỷ lệ bằng nhau, hàm chi phí có dạng cái bát tròn, đường đi đến cực tiểu nhanh gọn và thẳng.
- **Bên phải:** Đặc trưng 1 nhỏ hơn Đặc trưng 2, hàm chi phí có dạng bát kéo dài. Đường đi của thuật toán ngoằn ngoèo, tốn nhiều thời gian hội tụ.



Hình 4-7. Gradient Descent với và không có Scaling

Hạ Gradient theo lô (Batch Gradient Descent)

- Batch Gradient Descent (Hạ Gradient theo lô) tính toán đạo hàm riêng của hàm chi phí đối với mỗi tham số θ_j bằng cách sử dụng **toàn bộ tập dữ liệu huấn luyện** tại mỗi bước lặp.
- **Vector Gradient của hàm chi phí:**

$$\nabla_{\theta} MSE(\theta) = \frac{2}{m} \mathbf{X}^T (\mathbf{X}\theta - \mathbf{y}) \quad (5)$$

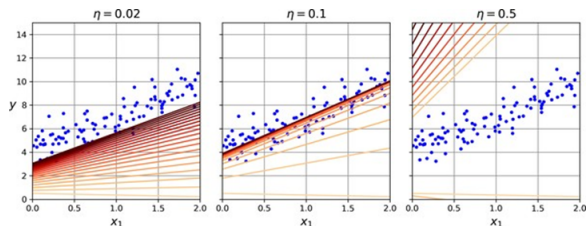
- **Bước nhảy Gradient Descent:**

$$\theta^{(\text{bước tiếp theo})} = \theta - \eta \nabla_{\theta} MSE(\theta) \quad (6)$$

- Mặc dù chậm trên tập dữ liệu rất lớn (do phải dùng toàn bộ dữ liệu), nhưng nó lại chia tỷ lệ tốt với số lượng đặc trưng lớn (tốt hơn Phương trình chuẩn tắc).

Minh họa: Batch GD với các tốc độ học khác nhau

- Hình bên cho thấy 10 bước đầu tiên của Gradient Descent theo lô.
- Trái ($\eta = 0.02$):** Tốc độ học quá thấp, sẽ đạt cực tiểu nhưng chậm.
- Giữa ($\eta = 0.1$):** Tốc độ học rất tốt, thuật toán hội tụ nhanh chóng chỉ trong vài bước.
- Phải ($\eta = 0.5$):** Tốc độ học quá cao, thuật toán nhảy loạn xạ và đang phân kỳ (diverging).



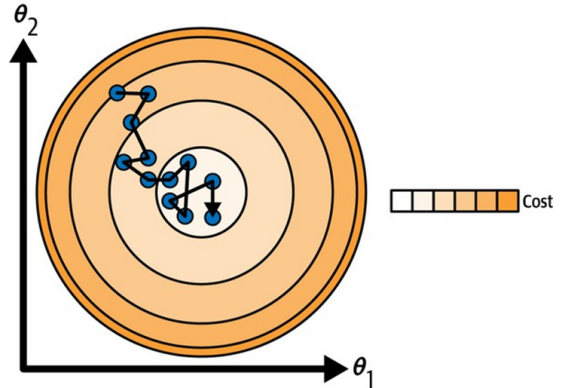
Hình 4-8. Batch GD với tốc độ học khác nhau

Hạ Gradient ngẫu nhiên (Stochastic Gradient Descent - SGD)

- Vấn đề chính của Batch GD là sử dụng toàn bộ tập dữ liệu huấn luyện để tính gradient tại mỗi bước, làm nó rất chậm với tập dữ liệu lớn.
- **Hạ Gradient ngẫu nhiên (SGD)** ngược lại, chọn ra một trường hợp ngẫu nhiên trong tập huấn luyện tại mỗi bước và chỉ tính gradient trên trường hợp duy nhất đó.
- Điều này làm cho thuật toán cực kỳ nhanh, và có khả năng thực hiện huấn luyện các mô hình rất lớn vì mỗi lần lặp chỉ cần 1 trường hợp trong bộ nhớ (hỗ trợ Out-of-core learning).

Đặc điểm đường đi của Stochastic Gradient Descent

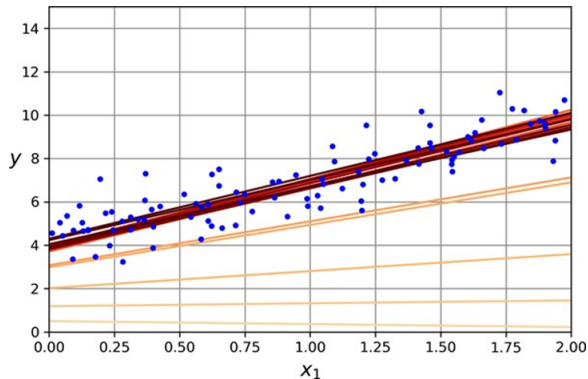
- Do tính chất ngẫu nhiên, SGD thất thường hơn nhiều so với Batch GD.
- Thay vì giảm nhẹ nhàng cho đến khi đạt mức tối thiểu, hàm chi phí sẽ dao động lên xuống, giảm dần về trung bình.
- Ngay cả khi kết thúc gần cực tiểu, các giá trị tham số sẽ tiếp tục dao động. Nghĩa là giá trị cuối cùng là tốt, nhưng chưa phải tối ưu tuyệt đối.
- Ưu điểm: Sự ngẫu nhiên giúp nó nhảy ra khỏi các cực tiểu cục bộ tốt hơn Batch GD.



Hình 4-9. Đặc điểm bất thường của SGD

Minh họa: 20 bước đầu tiên của SGD

- Hình bên cho thấy 20 bước đầu tiên của SGD.
- Bạn có thể thấy các bước không đều đặn như Batch GD. Để SGD hội tụ sát hơn, ta phải áp dụng kỹ thuật **lịch học (learning schedule)**: Tốc độ học lớn lúc đầu (tránh kẹt cực tiểu cục bộ), và giảm dần tốc độ học khi tiến về gần đích để ổn định tại đáy thung lũng.



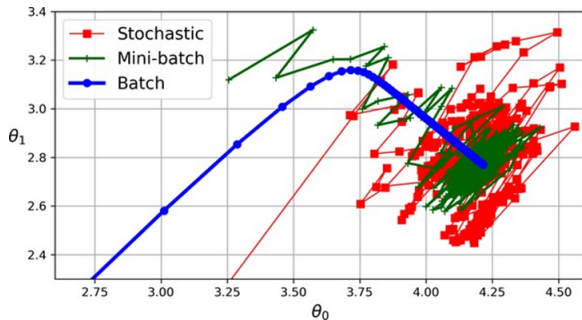
Hình 4-10. 20 bước đầu tiên của SGD

Hạ Gradient theo Mini-Batch

- Tại mỗi bước, thay vì tính toán gradient dựa trên toàn bộ dữ liệu (như Batch GD) hoặc chỉ một trường hợp (như SGD), thuật toán tính gradient trên các tập con nhỏ ngẫu nhiên gọi là **mini-batch**.
- Ưu điểm lớn nhất là bạn có thể tăng hiệu suất từ việc tối ưu hóa phần cứng của các phép toán ma trận, đặc biệt khi sử dụng GPUs.
- Sự tiến bộ của thuật toán ít thất thường hơn SGD (với mini-batch khá lớn). Nó sẽ tiến gần cực tiểu hơn SGD một chút, nhưng có thể khó thoát cực tiểu cục bộ hơn.

So sánh đường đi của 3 thuật toán Gradient Descent

- **Batch GD (Đỏ):** Đi đến thẳng cực tiểu và dừng lại, nhưng tốn nhiều thời gian mỗi bước.
- **SGD (Trắng):** Dao động mạnh xung quanh đích.
- **Mini-Batch GD (Xanh):** Ít dao động hơn SGD và tiến gần đích hơn, nhưng vẫn không dừng lại chính xác tại một điểm trừ khi có lịch học (learning schedule) tốt.



Hình 4-11. Đường đi của 3 thuật toán GD

Bảng so sánh tổng hợp các thuật toán hồi quy tuyến tính

Thuật toán	Lớn m (trường hợp)	Ngoài lõi	Lớn n (đặc trưng)	Hyperparams	Scaling	Scikit-Learn
Phương trình chuẩn tắc	Nhanh	Không	Chậm	0	Không	N/A
SVD	Nhanh	Không	Chậm	0	Không	LinearRegression
Batch GD	Chậm	Không	Nhanh	2	Có	N/A
Stochastic GD	Nhanh	Có	Nhanh	≥ 2	Có	SGDRegressor
Mini-batch GD	Nhanh	Có	Nhanh	≥ 2	Có	N/A

Lưu ý: Sau khi huấn luyện, tất cả các thuật toán này đều tạo ra mô hình rất giống nhau và đưa ra dự đoán theo cùng một cách.

3. Hồi quy đa thức, Đường cong học tập & Chính quy hóa

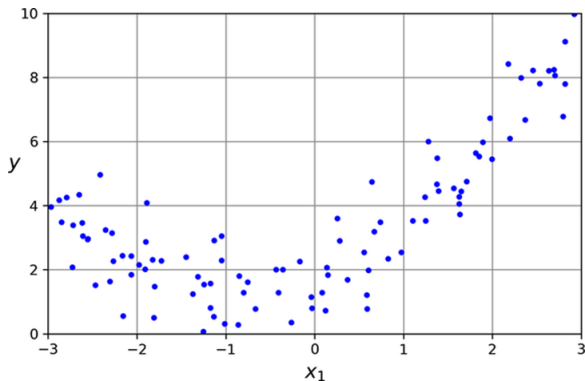
Hồi quy đa thức (Polynomial Regression)

- Điều gì sẽ xảy ra nếu dữ liệu của bạn phức tạp hơn một đường thẳng?
- Thật đáng ngạc nhiên, bạn có thể sử dụng một mô hình tuyến tính để khớp với dữ liệu phi tuyến tính.
- Cách đơn giản: Thêm các lũy thừa của mỗi đặc trưng làm đặc trưng mới, sau đó huấn luyện mô hình tuyến tính trên tập đặc trưng mở rộng này.
- Kỹ thuật này được gọi là **Hồi quy đa thức**.
- `PolynomialFeatures` trong Scikit-Learn có thể dễ dàng tạo ra các biến bậc cao và kết hợp biến (tương tác giữa các đặc trưng).

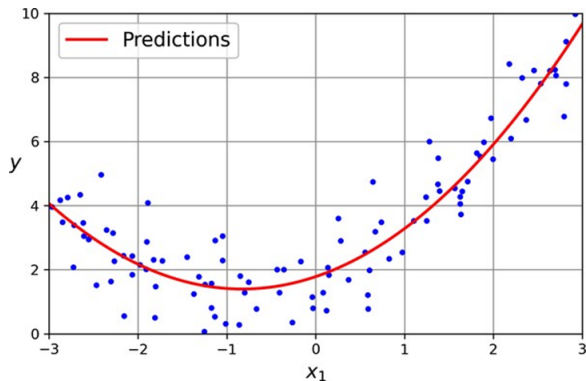
Minh họa: Khớp Hồi quy đa thức trên dữ liệu phi tuyến

- Giả sử chúng ta có một tập dữ liệu phân bố theo phương trình bậc hai ($y = ax^2 + bx + c + \text{nhiều}$). Một đường thẳng tuyến tính bình thường không bao giờ khớp được (Underfitting).
- Bằng cách dùng `PolynomialFeatures(degree=2)`, chúng ta thêm x^2 vào bộ đặc trưng. Sau đó dùng `LinearRegression` khớp dữ liệu mới.
- Kết quả thu được là một đường parabol bám rất sát dữ liệu (như Hình 4-13).

Minh họa: Dữ liệu phi tuyến tính và Dự đoán Hồi quy Đa thức



Hình 4-12. Tập dữ liệu phi tuyến tính ngẫu nhiên



Hình 4-13. Dự đoán mô hình hồi quy đa thức